
CreoAgent: A Self-Reflective Multimodal Agent for Autonomous Visual Creation with Dynamic Multi-Tool Orchestration

Anonymous Author(s)

Affiliation

Address

email

Abstract

Complex visual creation tasks—such as generating a scene, segmenting specific regions, compositing elements, and enhancing resolution—require the coordinated use of multiple specialized vision tools across several interaction steps. Existing tool-augmented multimodal agents are limited by static tool-use patterns learned from supervised demonstrations, blind decision-making without visual inspection of intermediate results, and the inability to manage multiple visual assets within a trajectory. In this paper, we propose CreoAgent, a self-reflective multimodal agent built on Qwen3-VL-8B that autonomously orchestrates seven heterogeneous vision tools (generation, editing, segmentation, grounding, super-resolution, cropping, and OCR) through multi-turn interaction. CreoAgent features two key designs: (i) vision-grounded adaptive reasoning, where the agent visually inspects intermediate results after each tool call and adjusts its plan through learned correction, tool switching, and rollback behaviors; and (ii) an Image Asset Dictionary that explicitly tracks all visual artifacts produced during a trajectory, enabling selective operations on specific intermediate images. To train CreoAgent, we construct CreoTraj, a large-scale dataset comprising 100K multi-turn tool-use trajectories with complete annotations and 10K curated RL instructions. We further design a multi-tool RL training framework based on GRPO with trajectory-level judge rewards and GPU-isolated asynchronous tool execution. Experiments on [benchmarks] demonstrate that CreoAgent achieves [superior/competitive] performance, with RL training yielding [XX.X]-point improvement over SFT alone and a [XX]% increase in multi-tool utilization rate.

1 Introduction

The ability to create and edit visual content is fundamental to a wide range of applications, from graphic design and advertising to social media and digital art. While recent advances in diffusion models and multimodal large language models (MLLMs) have made remarkable progress on individual visual tasks—such as text-to-image generation, instruction-based editing, and image super-resolution—complex visual creation often demands the *coordinated use of multiple specialized tools across several interaction steps*. Consider the task: "Generate a sunset beach scene, segment the sky region, replace it with a starry night, and enhance the resolution to 4K." Accomplishing this requires chaining at least four distinct operations (generation, segmentation, editing, super-resolution), each with its own model, while maintaining coherence across the entire workflow. No single end-to-end model can currently handle such diverse and compositional demands.

Tool-augmented multimodal agents [? ? ?] have emerged as a promising paradigm for bridging this gap: an LVLM-based agent interprets the user’s intent, selects appropriate tools, and orchestrates

their execution. However, existing approaches suffer from three critical limitations. **First**, most agents are trained via supervised fine-tuning (SFT) on pre-collected tool-use demonstrations [? ? ?], learning fixed invocation patterns that cannot adapt when intermediate results are unsatisfactory or when the task requires novel tool combinations. **Second**, agents relying on text-only chain-of-thought reasoning [?] decide subsequent tool calls without visually inspecting intermediate results. This *blind decision-making* leads to instruction hallucination, where the agent’s plan diverges from the actual visual state. **Third**, in multi-turn creation workflows, multiple intermediate images are produced (e.g., a generated base image, a segmented mask, an edited variant), yet existing agents lack an explicit mechanism to track, reference, and selectively operate on these visual assets, leading to confusion and error propagation.

Recent work has begun to address some of these challenges individually. JarvisArt [?] applies GRPO reinforcement learning for photo retouching within Adobe Lightroom, but is restricted to a single application. OpenThinkIMG [?] proposes V-ToolRL for learning adaptive tool invocation policies, but focuses on single-turn visual QA rather than multi-turn creative workflows. JarvisEvo [?] introduces interleaved multimodal chain-of-thought to combat instruction hallucination, but does not address multi-tool orchestration or asset management. Agent Banana [?] presents a multi-agent framework with layer-aware editing, but operates without policy optimization. None of these works simultaneously tackles dynamic multi-tool orchestration, vision-grounded adaptive reasoning, and explicit multi-asset management within a unified, RL-trained framework.

In this paper, we propose **CreoAgent**, a self-reflective multimodal agent that autonomously creates and edits images through dynamic multi-tool orchestration. Built on Qwen3-VL-8B, CreoAgent integrates seven heterogeneous vision tools spanning generation, editing, segmentation, grounding, super-resolution, cropping, and OCR. At its core are two key designs: (i) a **vision-grounded adaptive reasoning** mechanism, where the agent visually inspects intermediate results after each tool call and adjusts its plan accordingly—enabling learned behaviors such as parameter correction, tool switching, and rollback; and (ii) an **Image Asset Dictionary**, a structured registry that explicitly tracks all visual artifacts produced during a trajectory, allowing the agent to selectively reference and operate on any intermediate image by its semantic identifier. CreoAgent is trained through a two-stage pipeline: SFT on CreoTraj-SFT (100K trajectories) for bootstrapping basic tool-use capability, followed by GRPO reinforcement learning with a trajectory-level judge reward on CreoTraj-RL (10K instructions) for learning dynamic multi-tool strategies.

Our main contributions are as follows:

1. **We propose CreoAgent, a self-reflective multimodal agent for autonomous visual creation** that dynamically orchestrates 7+ heterogeneous vision tools through multi-turn interaction, featuring vision-grounded adaptive reasoning and an Image Asset Dictionary for explicit multi-asset management. To our knowledge, CreoAgent is the first RL-trained agent that performs dynamic multi-tool orchestration for complex visual creation tasks.
2. **We construct CreoTraj**, a large-scale multi-turn multi-tool trajectory dataset comprising 100K SFT trajectories—each containing user instructions, chain-of-thought reasoning, structured tool invocations, and tool execution results including intermediate images—and 10K curated RL instructions. Built through a semi-automatic pipeline, CreoTraj is the largest and most comprehensively annotated dataset for multi-tool visual agent training.
3. **We design a multi-tool RL training framework** based on GRPO with trajectory-level judge rewards, supporting asynchronous multi-tool execution with GPU-isolated tool workers. Extensive experiments demonstrate that CreoAgent achieves [superior/competitive] performance on [benchmarks], with RL training yielding substantial improvements in multi-tool coordination over SFT alone.

2 Related Work

2.1 Multimodal Large Language Models

The rapid progress in large vision-language models (LVLMs) has established a powerful foundation for multimodal agents. LLaVA [?] pioneered the visual instruction tuning paradigm, demonstrating that aligning a vision encoder with an LLM through instruction-following data yields strong visual understanding. Subsequent work has significantly expanded this capability: Qwen2-VL [?] introduced

dynamic resolution processing that handles images at arbitrary aspect ratios, making it particularly suitable as an agent backbone for visual tasks. InternLM-XComposer [?] and InternVL [?] further advanced multimodal reasoning with improved vision-language alignment strategies. More recently, unified multimodal models such as Janus-Pro [?] and BAGEL [?] have demonstrated the feasibility of combining visual understanding and generation within a single architecture. While these models exhibit increasingly strong visual perception and reasoning, they do not inherently possess the ability to interact with external tools or execute multi-step visual creation workflows—a gap that our work addresses.

2.2 Multimodal Agents for Visual Creation

Building autonomous agents for visual creation and editing has become an active research frontier. Early efforts such as HuggingGPT [?] and Visual ChatGPT [?] employed LLMs as high-level planners to coordinate pre-existing vision foundation models, relying on prompt engineering and fixed task decomposition pipelines. GenArtist [?] advanced this paradigm by introducing a planning tree with self-correction for unified image generation and editing, though it still uses a text-only LLM as the coordinator without visual perception of intermediate results.

More recent systems have moved toward tighter integration of vision-language backbones with editing capabilities. JarvisArt [?] trains a multimodal agent to emulate professional retouching artists within Adobe Lightroom, supporting over 200 operations via its A2L protocol, but is limited to a single application environment. JarvisEvo [?] addresses the problem of instruction hallucination by proposing interleaved multimodal chain-of-thought (iMCoT) and co-evolutionary training (SEPO), though its focus remains on single-tool editing quality rather than multi-tool orchestration. Agent Banana [?] introduces a hierarchical planner-executor framework with Context Folding and Image Layer Decomposition for high-fidelity multi-turn editing, but operates as a training-free pipeline without policy optimization. Similarly, RefineEdit-Agent [?] proposes a training-free closed-loop editing framework with iterative refinement.

ImageEdit-R1 [?] represents a concurrent effort that uses RL to coordinate multiple specialized agents for image editing, treating it as a sequential decision-making problem. However, its multi-agent architecture separates decomposition, sequencing, and editing into distinct models, whereas our approach uses a single unified agent that autonomously orchestrates diverse tools through adaptive reasoning. A key distinction of CreoAgent is the combination of (i) a unified LVLM backbone that perceives intermediate visual results, (ii) an explicit Image Asset Dictionary for managing multiple visual artifacts across turns, and (iii) RL-trained dynamic multi-tool orchestration spanning the full visual creation pipeline—from generation to perception to enhancement.

2.3 Tool-Augmented Visual Reasoning

Augmenting LVLMs with external tools has emerged as a promising strategy for overcoming the inherent limitations of end-to-end models. LLaVA-Plus [?] extends LLaVA with tool-use capabilities by training on tool-use demonstrations, while CogCom [?] and MLLM-Tool [?] further explore supervised approaches for learning when and how to invoke visual tools. However, these SFT-based methods are constrained by the quality and coverage of their demonstration data, limiting generalization to novel tool combinations.

OpenThinkIMG [?] makes a significant step forward by proposing V-ToolRL, a reinforcement learning framework where the agent learns adaptive tool invocation policies through direct interaction with tool environments. It also provides a unified tool registry with standardized interfaces for heterogeneous vision tools. However, OpenThinkIMG primarily targets single-turn visual question answering tasks (e.g., chart reasoning) rather than multi-turn creative workflows.

Tool-MVR [?] introduces meta-verification and reflection learning for tool-augmented LLMs, constructing a high-quality tool-use dataset through systematic validation. ARTIST [?] proposes an RL-based framework for learning optimal tool-use strategies, demonstrating that cold-start RL without supervised trajectories can outperform SFT baselines on mathematical reasoning. These works demonstrate the effectiveness of RL for tool learning but operate in text-only domains. CreoAgent extends this paradigm to the multimodal visual creation setting, where tool interactions involve heterogeneous vision models and the agent must reason over both textual and visual feedback.

2.4 Reinforcement Learning for Agent Training

Reinforcement learning has become an increasingly important training paradigm for LLM-based agents. The ReAct framework [?] established the foundation by interleaving reasoning and acting, though it relies on prompting rather than learned policies. DeepSeek-R1 [?] demonstrated that RL with verifiable rewards can incentivize strong reasoning capabilities in LLMs, inspiring a wave of RL-based agent training methods.

In the tool-use domain, ToolRL [?] proposes a principled reward design framework addressing the challenge that coarse-grained answer-matching rewards are insufficient for learning fine-grained tool-use strategies. ReTool [?] combines cold-start data synthesis with iterative RL for code-enhanced reasoning. For visual generation and editing specifically, JarvisArt [?] introduces GRPO-R, a variant of GRPO with customized retouching rewards, while JarvisEvo [?] proposes SEPO, a co-evolutionary framework where the editor and evaluator policies improve jointly to prevent reward hacking. Edit-R1 [?] builds a reasoning-based reward model for image editing and applies GRPO to optimize editing models directly. EARL [?] investigates RL with MLLM-based verifiers for autoregressive image editing, finding that strong vision-language models serve as effective reward signals.

While these works demonstrate the promise of RL for visual tasks, they predominantly focus on single-tool settings (e.g., optimizing a single editing model’s outputs). CreoAgent introduces a distinct challenge: learning a multi-tool orchestration policy where the agent must decide *which* tool to invoke, *with what parameters*, and *on which image asset*—all within a multi-turn trajectory. Our trajectory-level judge reward and asynchronous multi-tool training framework are designed specifically for this more complex action space.

3 Method

3.1 Overview

Given a natural language instruction Q describing a complex visual creation or editing task (e.g., "Generate a sunset beach scene, segment the sky region, replace it with a starry night, and enhance the final image to 4K resolution"), CreoAgent aims to produce a high-quality visual output by autonomously orchestrating multiple specialized vision tools across several interaction turns. As illustrated in Figure ??, CreoAgent consists of four key components: (1) a **vision-language backbone** (Qwen3-VL-8B) that jointly processes visual and textual inputs for perception and reasoning; (2) a **structured reasoning and tool invocation interface** that enables the agent to express chain-of-thought reasoning and emit structured tool calls; (3) an **Image Asset Dictionary** that explicitly tracks all visual artifacts produced during the trajectory; and (4) a **two-stage training pipeline** (SFT $\rightarrow$ GRPO RL) that first bootstraps basic tool-use capability and then optimizes multi-tool orchestration through trajectory-level reinforcement learning. In the following subsections, we detail each component.

3.2 Agent Architecture

3.2.1 Structured Reasoning and Tool Invocation

At each interaction turn t , CreoAgent receives the current context—comprising the user instruction Q , the dialogue history $H_{<t}$, and visual observations from the Image Asset Dictionary—and generates a response in the following structured format:

$$a_t = \langle \text{reason} \rangle r_t \langle / \text{reason} \rangle \langle \text{tool_call} \rangle f_t(p_t) \langle / \text{tool_call} \rangle$$

where r_t is the chain-of-thought reasoning text that articulates the agent’s analysis of the current visual state, its plan for the next step, and the justification for the chosen tool; $f_t \in \mathcal{T}$ is the selected tool from the tool set $\mathcal{T}$; and p_t denotes the tool parameters. This explicit separation of reasoning from action enables interpretable decision-making and facilitates RL training by providing a clear structure for credit assignment.

Tool	Function	Backend
ImageGeneration	Text-to-image synthesis	FLUX.2 Klein
ImageEdit	Instruction-based image editing	FLUX.2 Klein
ImageCrop	Region-of-interest cropping	PIL
ImageGrounding	Open-vocabulary object localization	GroundingDINO
ImageSAM	Segmentation mask extraction	SAM
ImageSR	Image super-resolution	RealESRGAN
OCR	Text recognition in images	–

After execution, the tool returns an observation o_t (which may include a newly generated or modified image), and the agent proceeds to the next turn with an updated context. The agent may also choose to terminate the trajectory by emitting a special `Terminate` action when it judges the task to be complete. The maximum number of interaction turns is set to $K = 6$.

3.2.2 Tool Set

CreoAgent integrates a diverse set of seven vision tools spanning the full visual creation pipeline:

Table. Tool set of CreoAgent.

This tool set covers the core capabilities needed for complex visual creation: generation, editing, spatial perception (grounding, segmentation), enhancement (super-resolution), and text understanding (OCR). Unlike JarvisArt [?], which is restricted to operations within a single application (Lightroom), CreoAgent can compose arbitrary tool sequences to accomplish tasks that no single tool could handle alone.

3.2.3 Image Asset Dictionary

A central challenge in multi-turn visual creation is managing the growing number of intermediate images. When an agent generates, edits, crops, or segments images across multiple turns, it must keep track of which image is which and selectively operate on specific assets. Existing agents lack an explicit mechanism for this, leading to confusion and error propagation—for instance, applying an edit to the wrong intermediate image or losing track of a useful intermediate result.

We address this with the **Image Asset Dictionary** $\mathcal{A}$, a structured registry that maintains all visual assets produced during a trajectory. Each time a tool execution produces a new image, it is registered in $\mathcal{A}$ with a unique identifier following a semantic naming convention:

$$\text{id} = \langle \text{operation} \rangle_ \langle \text{index} \rangle$$

For example, `generation_0` refers to the first generated image, `extract_1` refers to the second extracted region, and `edit_2` refers to the third editing result. The dictionary maps each identifier to the corresponding image object:

$$\mathcal{A} = \{\text{generation}_0 \mapsto I_0, \text{extract}_1 \mapsto I_1, \text{edit}_2 \mapsto I_2, \dots\}$$

At each reasoning step, the agent can reference any asset in $\mathcal{A}$ by its identifier when specifying tool parameters. For instance, the agent may emit:

```
<reason>The generated beach scene (generation_0) has a good
composition, but the sky needs replacement. I will first segment
the sky region from generation_0 using ImageSAM.</reason>
<tool_call>ImageSAM(image=generation_0, prompt="sky")</tool_call>
```

This mechanism provides three key benefits: (1) **Selective operation**: the agent can choose to operate on any specific intermediate result, not just the most recent one; (2) **Rollback capability**: if a later editing step degrades quality, the agent can reference an earlier asset (e.g., revert to `generation_0` instead of continuing from a flawed `edit_1`); (3) **Parallel asset management**: the agent can

maintain multiple independent visual artifacts (e.g., a mask and a base image) and compose them in a subsequent step.

3.2.4 Vision-Grounded Adaptive Reasoning

Unlike ReAct-style agents [?] that commit to a fixed plan at the outset, CreoAgent operates in a closed perception-action loop: after each tool execution, the agent *visually inspects* the result through its LVLM backbone before deciding the next action. This enables three adaptive behaviors that emerge through RL training:

- **Correction:** If the current result deviates from the instruction (e.g., incorrect color tone after editing), the agent re-invokes the same tool with adjusted parameters.
- **Tool switching:** If the current tool is unsuitable (e.g., editing cannot achieve the desired effect), the agent switches to an alternative tool.
- **Rollback:** If a sequence of edits has drifted from the desired outcome, the agent references an earlier asset in the Image Asset Dictionary and re-plans from that checkpoint.

Critically, these adaptive behaviors are not hard-coded rules but are *learned through reinforcement learning*. The RL training process (Section 3.4.2) encourages the agent to discover these strategies by rewarding trajectories that produce high-quality final outputs, regardless of the specific path taken.

3.3 CreoTraj: Multi-Turn Tool-Use Trajectory Dataset

To train CreoAgent, we construct **CreoTraj**, a large-scale dataset of multi-turn tool-use trajectories. CreoTraj consists of two subsets designed for different training stages:

3.3.1 CreoTraj-SFT (100K Trajectories)

Each SFT trajectory contains the complete interaction record:

$$\tau_{\text{sft}} = (Q, [(r_1, f_1, p_1, o_1), (r_2, f_2, p_2, o_2), \dots])$$

where Q is the user instruction, and each tuple (r_t, f_t, p_t, o_t) represents the chain-of-thought reasoning, tool selection, tool parameters, and tool execution result (including any generated images) at turn t . These trajectories primarily consist of single-turn interactions (average 1.2 tool calls per trajectory), establishing the agent’s foundational capability to understand instructions, reason about tool selection, and produce well-formatted tool calls.

Construction pipeline. We adopt a semi-automatic approach:

1. **Instruction curation:** We collect diverse visual creation instructions spanning multiple task categories (generation, editing, enhancement, composition, etc.).
2. **Trajectory generation:** A strong teacher model generates candidate trajectories by reasoning about the instruction and producing tool calls. Tools are executed to obtain real visual outputs.
3. **Human filtering and correction:** Human annotators review generated trajectories, filtering out low-quality or incorrect samples and correcting reasoning chains or tool parameters where needed. This ensures both the reasoning quality and the visual output quality of the training data.

This semi-automatic pipeline balances scalability (model-generated trajectories at scale) with quality (human verification ensures correctness).

3.3.2 CreoTraj-RL (10K Instructions)

For the RL training stage, we curate 10K high-quality user instructions that specifically require multi-tool coordination. Unlike the SFT data, these instructions are designed to be challenging: they often involve sequential operations (e.g., "generate → segment → edit → enhance"), compositional requirements (e.g., "create an image with specific objects in specific positions"), or quality-critical

tasks where the agent must iterate to achieve satisfactory results. Only the instructions are provided; the agent must discover effective tool-use strategies through exploration and reward feedback.

3.4 Two-Stage Training Pipeline

3.4.1 Stage 1: Supervised Fine-Tuning (Cold-Start)

We first perform supervised fine-tuning on CreoTraj-SFT to bootstrap the agent’s basic capabilities. Using the standard next-token prediction objective:

$$\mathcal{L}_{\text{sft}} = -\mathbb{E}_{\tau \sim \mathcal{D}_{\text{sft}}} \left[\sum_t \log \pi_{\theta}(a_t \mid Q, H_{<t}) \right]$$

where π_{θ} is the agent policy parameterized by θ , a_t is the ground-truth action (reasoning + tool call) at turn t , and $H_{<t}$ is the dialogue history up to turn t . This stage teaches the agent three fundamental skills: (1) understanding the structured output format (`<reason>...<tool_call>...`), (2) mapping instructions to appropriate tool selections, and (3) generating valid tool parameters.

Since the SFT data primarily consists of single-turn trajectories, the agent after this stage can reliably invoke individual tools but lacks the ability to orchestrate multi-tool sequences or adaptively adjust its strategy based on visual feedback. This limitation motivates the RL stage.

3.4.2 Stage 2: Multi-Tool RL with Trajectory-Level Judge Reward

To enable the agent to learn dynamic multi-tool orchestration, we employ Group Relative Policy Optimization (GRPO) [?] with a trajectory-level judge reward.

GRPO formulation. For each instruction Q_i from CreoTraj-RL, we sample a group of G complete trajectories $\{\tau_i^1, \tau_i^2, \dots, \tau_i^G\}$ from the current policy π_{θ} . Each trajectory involves the agent interacting with real tools over up to $K = 6$ turns, producing intermediate and final images. The trajectories are scored by a reward function $R(\cdot)$, and the policy is updated using the group-relative advantage:

$$\hat{A}_i^j = \frac{R(\tau_i^j) - \text{mean}(\{R(\tau_i^k)\}_{k=1}^G)}{\text{std}(\{R(\tau_i^k)\}_{k=1}^G) + \epsilon}$$

The policy gradient objective is:

$$\mathcal{L}_{\text{grpo}} = -\mathbb{E}_{i,j} \left[\hat{A}_i^j \cdot \log \pi_{\theta}(\tau_i^j \mid Q_i) - \beta \cdot D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right]$$

where π_{ref} is the reference policy (the SFT checkpoint) and β controls the KL penalty.

Trajectory-level judge reward. Unlike prior work that scores individual tool calls [?] or uses pixel-level metrics, we design a holistic trajectory-level reward. A judge model receives the complete trajectory text (the full chain of reasoning and tool calls) together with the final output image, and produces a single quality score $R \in [0, 1]$:

$$R(\tau) = \text{Judge}(\text{trajectory_text}(\tau), I_{\text{final}}(\tau))$$

This design has two important properties:

1. **Holistic evaluation:** The judge assesses both the quality of the final output *and* the coherence of the reasoning process, capturing whether the agent’s tool-use strategy is sensible.
2. **Flexibility:** Since the reward does not prescribe a specific tool sequence, the agent is free to discover novel strategies—including multi-step plans and rollback behaviors—as long as the final outcome is good. This is crucial for multi-tool settings where multiple valid paths exist for the same task.

To prevent reward hacking, trajectories where the agent immediately terminates without invoking any tool receive a near-zero score, discouraging the collapse to trivial solutions.

3.4.3 Asynchronous Multi-Tool Execution with GPU Isolation

Training a multi-tool RL agent presents a unique systems challenge: during each rollout, the agent must interact with real vision tools (diffusion models, segmentation networks, etc.) that require GPU resources. Naively co-locating tool execution with model training leads to GPU memory contention and out-of-memory errors.

We address this through a **GPU-isolated asynchronous architecture** built on Ray:

- **Training GPUs** ($N_{\text{train}} = 6$): Dedicated to the actor, reference model, and rollout inference. These GPUs are managed by the verl framework for colocated GRPO training.
- **Tool GPUs** ($N_{\text{tool}} = 2$): Dedicated to tool execution workers. GPU 6 hosts ImageGeneration and ImageSR workers; GPU 7 hosts ImageEdit, ImageGrounding, and ImageSAM workers. Tool workers are implemented as Ray remote actors with explicit GPU pinning (`num_gpus=0` with manual `CUDA_VISIBLE_DEVICES` assignment) to avoid conflicts with Ray’s GPU scheduling.

During rollout, the agent generates actions sequentially, but tool executions across different trajectories in a batch are dispatched **asynchronously**: when one trajectory is waiting for a tool response, other trajectories can proceed with their generation or tool calls. This overlapping execution significantly reduces the wall-clock time of each training step.

3.5 Implementation Details

We use Qwen3-VL-8B as the base model. The SFT stage trains for [X] epochs with a learning rate of [X] and batch size of [X]. For the RL stage, we set train batch size to 6, rollout group size $G = 2$, maximum prompt length to 8192 tokens, and maximum response length to 28672 tokens. The PPO mini-batch size is 6 with a micro-batch size of 1 per GPU. The maximum number of assistant turns is $K = 6$. All experiments are conducted on a node with 8 NVIDIA GPUs (6 for training, 2 for tools). The trajectory judge uses [judge model name] with a timeout of 180 seconds and up to 2 retries per evaluation. We set the fallback score to 0.3 for judge failures to maintain training stability.

4 Experiments

4.1 Experimental Setup

4.1.1 Evaluation Benchmarks

We evaluate CreoAgent on two benchmarks:

CreoTraj-Test. We hold out a test set from CreoTraj comprising [X] instructions that require multi-tool coordination. These instructions span diverse task categories including compositional generation (e.g., "generate an image of a cat on a beach, then segment the cat and place it on a mountain background"), iterative refinement (e.g., "generate a portrait, enhance the resolution, and crop the face region"), and complex editing chains. Each instruction is designed to require at least two distinct tool invocations. This benchmark directly evaluates the agent’s multi-tool orchestration capability.

[Public Benchmark]. To assess generalizability beyond our own data distribution, we additionally evaluate on [T2I-CompBench / MagicBrush / other public benchmark]. [Brief description of the benchmark and why it is relevant.]

4.1.2 Baselines

We compare CreoAgent against three carefully chosen baselines plus our full model, where each baseline corresponds to one core research question in our study:

- **Qwen3-VL-8B (zero-shot w/ tools):** The base vision-language model without task-specific training, prompted with the same tool descriptions and output format as CreoAgent. This baseline is used to verify whether explicit SFT/RL training is necessary for reliable multi-tool orchestration.

- **QwenImage-Edit**: A strong end-to-end image editing model that directly transforms an input image according to a text instruction without external tool orchestration. This baseline tests whether a multi-tool agent is more advantageous than a single editing model on complex compositional tasks that require grounding, segmentation, OCR, cropping, or super-resolution in addition to editing.
- **CreoAgent-SFT**: Our model after Stage-1 supervised fine-tuning on CreoTraj-SFT, without RL training. Comparing this variant with the full CreoAgent isolates the gain from RL-based policy optimization.
- **CreoAgent (full)**: The complete model after both SFT and GRPO RL training.

4.1.3 Evaluation Metrics

We organize our evaluation into **main outcome metrics** and **trajectory behavior metrics**. The main tables only report the former, while the latter are moved to a separate behavior analysis table to keep the core comparison compact and easier to read.

Main outcome metrics.

- **Instruction Following** (S_{inst}): a judge score from 1 to 10 measuring whether the final output faithfully satisfies the user instruction.
- **Visual Quality** (S_{qual}): a judge score from 1 to 10 measuring image fidelity, naturalness, and artifact level.
- **Overall Score** (S_{overall}): the mean of the two final-output scores, i.e., $S_{\text{overall}} = \frac{1}{2}(S_{\text{inst}} + S_{\text{qual}})$.
- **Task Completion Rate** (CR): the percentage of test cases where the system produces a valid final image and ends normally without crashing, looping, or returning an empty result.
- **Tool Accuracy** (Acc_{tool}): among all generated tool calls, the percentage whose selected tool type is appropriate for the current sub-task, judged by the evaluator model or rule-based matching when the expected tool is unambiguous.
- **Avg. Tool Calls**: the average number of tool invocations per trajectory, used as a lightweight efficiency indicator.

Trajectory behavior metrics. These metrics are not included in the main comparison table because they are only defined for tool-agent trajectories and are mainly used to explain *why* RL improves over SFT.

- **Multi-Tool Rate** (MTR): the percentage of trajectories that invoke at least two distinct tools.
- **Invalid Tool Call Rate** (ITCR): the percentage of tool calls that are malformed, contain invalid arguments, or reference a nonexistent image asset id.
- **Premature Termination Rate** (PTR): the percentage of trajectories that emit Terminate before satisfying the major requirements in the instruction.
- **Reasoning Coherence** (S_{reas}): an optional judge score for analyzing whether the trajectory-level reasoning and tool sequence are logically consistent. We keep it for ablations and qualitative analysis, but do not use it as a primary ranking metric in the main result table because it is more subjective than final-output quality and completion rate.

4.2 Main Results

4.2.1 Overall Comparison

Table 1 presents the main comparison on CreoTraj-Test. The experiment is designed to support three conclusions.

Training is necessary for reliable tool use. CreoAgent (full) substantially outperforms Qwen3-VL-8B (zero-shot w/ tools) in both final quality and task completion, while also achieving higher Acc_{tool} with a reasonable number of tool calls. This confirms that prompting a pretrained LVLM alone is insufficient for stable multi-turn tool orchestration.

Table 1: Main results on CreoTraj-Test. The main table is intentionally compact and focuses on final output quality, task completion, tool selection correctness, and tool-use cost. Tool-specific metrics for QwenImage-Edit are marked as N/A because it does not expose explicit tool trajectories.

Method	S_{inst}	S_{qual}	S_{overall}	CR	Acc_{tool}	Avg. Calls
Qwen3-VL-8B (zero-shot w/ tools)	X.X	X.X	X.X	X.X%	X.X%	X.X
QwenImage-Edit	X.X	X.X	X.X	X.X%	N/A	N/A
CreoAgent-SFT	X.X	X.X	X.X	X.X%	X.X%	X.X
CreoAgent (full)	X.X	X.X	X.X	X.X%	X.X%	X.X

Table 2: Results on the Complex Multi-Tool Subset of CreoTraj-Test. This subset is designed to highlight tasks where a single end-to-end editor is structurally insufficient because the instruction requires explicit perception, region selection, or enhancement operations.

Method	S_{inst}	S_{overall}	CR
QwenImage-Edit	X.X	X.X	X.X%
CreoAgent-SFT	X.X	X.X	X.X%
CreoAgent (full)	X.X	X.X	X.X%

Multi-tool agents are more suitable than single editing models for complex compositional tasks. Compared with QwenImage-Edit, CreoAgent (full) achieves higher instruction-following and task completion on instructions that require explicit perception and post-processing operations such as grounding, segmentation, OCR, cropping, and super-resolution. This suggests that decomposing a complex request into multiple specialized tools is more effective than relying on a single end-to-end editor when the task goes beyond local editing.

RL improves over SFT by learning better long-horizon decisions. CreoAgent (full) outperforms CreoAgent-SFT by [X.X] points in S_{overall} and improves CR with a similar or more effective tool-call budget, indicating that RL does not merely polish image quality but also improves trajectory-level decision making.

To make the comparison with QwenImage-Edit more diagnostic, we additionally report results on a **Complex Multi-Tool Subset** of CreoTraj-Test, where each instruction explicitly requires at least two heterogeneous operations beyond plain editing (e.g., grounding+editing, segmentation+editing+SR, OCR-guided editing, crop+SR).

4.2.2 Multi-Tool Behavior Analysis

The main tables above intentionally hide trajectory-specific process metrics. To explain where the RL gain comes from, we separately compare SFT and RL policies using behavior statistics computed from tool-agent trajectories only.

We expect RL to increase MTR, reduce ITCR and PTR, and improve S_{reas} , which would directly support our claim that RL encourages more complete multi-step execution and better self-corrective tool use than SFT alone.

4.2.3 Results on [Public Benchmark]

[Placeholder for public benchmark results. This section demonstrates generalizability beyond CreoTraj-Test.]

4.3 Ablation Studies

To isolate the contribution of each design choice, we conduct ablation experiments on CreoTraj-Test.

4.3.1 Effect of Image Asset Dictionary

In the variant without the Image Asset Dictionary, the agent can only operate on the most recent image (i.e., no explicit identifier-based referencing). Table 4 shows that removing the dictionary

Table 3: Behavior analysis on CreoTraj-Test. These metrics are only computed for agent trajectories and therefore are reported separately from the main model comparison.

Method	MTR	ITCR	PTR	Avg. Calls	S_{reas}
Qwen3-VL-8B (zero-shot w/ tools)	X.X%	X.X%	X.X%	X.X	X.X
CreoAgent-SFT	X.X%	X.X%	X.X%	X.X	X.X
CreoAgent (full)	X.X%	X.X%	X.X%	X.X	X.X

Table 4: Ablation on the Image Asset Dictionary.

Variant	S_{overall}	Acc_{tool}	CR	MTR
CreoAgent (full)	X.X	X.X%	X.X%	X.X%
w/o Image Asset Dictionary	X.X	X.X%	X.X%	X.X%

leads to a [X.X]-point drop in overall score and a significant decrease in task completion rate. Qualitative analysis reveals that without the dictionary, the agent frequently applies edits to the wrong intermediate image or loses track of useful earlier results, especially in trajectories involving ≥ 3 tool calls.

4.3.2 Effect of Trajectory-Level Judge Reward

We compare our trajectory-level judge reward against two alternatives: (1) a *final-image-only* judge that scores only the final output image without seeing the trajectory text, and (2) a *step-level tool accuracy* reward that scores each individual tool call independently. Table 5 shows that our trajectory-level design achieves the best results. The final-image-only variant produces lower reasoning coherence scores because the reward provides no signal about the quality of intermediate reasoning. The step-level reward leads to lower multi-tool rates, as the agent learns to optimize individual steps rather than holistic trajectories, often defaulting to safe single-tool solutions.

4.3.3 Effect of RL Training Data Scale

We vary the number of RL training instructions from 1K to 10K. Table 6 shows that performance scales with RL data size, with the most significant jump from 1K to 5K. This suggests that a diverse set of multi-tool instructions is important for the agent to learn generalizable orchestration strategies.

4.3.4 SFT Data Scale Analysis

We examine how SFT data scale affects both the SFT checkpoint quality and the downstream RL training. Table 7 shows that while more SFT data consistently improves the SFT-only model, the RL gain (Δ) is relatively stable across scales, suggesting that RL can compensate for limited SFT data to some extent. Nevertheless, the best absolute performance is achieved with the full 100K SFT data.

4.4 Qualitative Analysis

4.4.1 Multi-Tool Orchestration Examples

Figure ?? presents representative examples where CreoAgent successfully orchestrates multiple tools across several turns. In Example 1, the agent receives the instruction "[example instruction]" and autonomously executes a four-step pipeline: [generation $\rightarrow$ segmentation $\rightarrow$ editing $\rightarrow$ super-resolution], inspecting intermediate results at each step and adjusting parameters when needed. We highlight how the Image Asset Dictionary enables the agent to selectively reference earlier results—for instance, reverting to `generation_0` after an unsatisfactory `edit_1`.

4.4.2 Adaptive Behavior Analysis

We qualitatively analyze the three types of adaptive behaviors described in Section 3.2.4:

Correction. [Example where the agent re-invokes the same tool with adjusted parameters after observing an unsatisfactory result.]

Table 5: Ablation on reward design for RL training.

Reward Design	S_{overall}	S_{inst}	S_{qual}	MTR
Trajectory-level judge (ours)	X.X	X.X	X.X	X.X%
Final-image-only judge	X.X	X.X	X.X	X.X%
Step-level tool accuracy	X.X	X.X	X.X	X.X%

Table 6: Effect of RL instruction set size.

RL Instructions	S_{overall}	MTR	Avg. Calls
1K	X.X	X.X%	X.X
5K	X.X	X.X%	X.X
10K (full)	X.X	X.X%	X.X

Tool switching. [Example where the agent switches from one tool to another after determining the initial choice was suboptimal.]

Rollback. [Example where the agent references an earlier asset in the Image Asset Dictionary to recover from a failed editing sequence.]

These behaviors emerge primarily after RL training; the SFT-only model rarely exhibits correction or rollback behaviors, instead following a linear tool-call sequence.

4.4.3 Failure Cases

We also present representative failure cases to illustrate the current limitations of CreoAgent:

- **Ambiguous instructions:** When the user instruction is vague (e.g., "make the image better"), the agent may select a plausible but incorrect tool chain.
- **Tool capability limits:** Some tasks exceed the capabilities of the available tools (e.g., fine-grained style transfer requiring tools not in the current set).
- **Error accumulation:** In very long trajectories (≥ 5 turns), errors can accumulate despite the rollback mechanism, as the agent may not recognize subtle quality degradation.

4.5 Training Efficiency Analysis

Table 8 reports the training cost of each stage. The GPU-isolated asynchronous architecture (Section 3.4.3) reduces the RL training wall-clock time by approximately [X]% compared to a naive synchronous baseline, by overlapping tool executions across trajectories in a batch.

5 Conclusion

We have presented CreoAgent, a self-reflective multimodal agent that autonomously creates and edits images by dynamically orchestrating multiple heterogeneous vision tools through multi-turn interaction. CreoAgent introduces two key technical designs—vision-grounded adaptive reasoning and the Image Asset Dictionary—that together enable the agent to inspect intermediate results, adjust its plan in real time, and manage multiple visual assets throughout a trajectory. Supported by CreoTraj, a large-scale dataset of 100K multi-turn tool-use trajectories and 10K curated RL instructions, and trained via a two-stage pipeline combining SFT with GRPO reinforcement learning using trajectory-level judge rewards, CreoAgent demonstrates [strong/state-of-the-art] performance on [benchmarks], substantially outperforming the SFT-only baseline in multi-tool coordination.

Limitations. Our work has several limitations that suggest directions for future research. First, CreoAgent’s tool set is currently fixed at seven tools; extending the framework to support dynamic tool discovery and zero-shot adoption of new tools would broaden its applicability. Second, the trajectory-level judge reward, while effective, relies on a strong external MLLM, introducing both computational cost and potential bias from the judge model. Exploring self-evaluation or learned reward models, as in SEPO [?], could mitigate this dependency. Third, our RL training requires

Table 7: Effect of SFT data scale on downstream RL performance.

SFT Trajectories	S_{overall} (SFT-only)	S_{overall} (after RL)	Δ
10K	X.X	X.X	+X.X
50K	X.X	X.X	+X.X
100K (full)	X.X	X.X	+X.X

Table 8: Training efficiency comparison.

Metric	SFT Stage	RL Stage
Training time	[X] hours	[X] hours
GPU resources	$6 \times [\text{GPU type}]$	$6 (\text{train}) + 2 (\text{tools}) \times [\text{GPU type}]$
Avg. rollout time per step	–	[X] seconds
Tool execution overhead	–	[X]% of total step time

489 real tool execution during rollouts, which is computationally expensive despite our asynchronous
 490 architecture; more efficient exploration strategies or model-based RL approaches could reduce this
 491 cost.

492 **Future work.** We plan to extend CreoAgent in several directions: (i) scaling to a broader and
 493 dynamically expandable tool set, (ii) incorporating user feedback for interactive refinement during
 494 deployment, (iii) exploring self-improvement mechanisms where the agent generates its own training
 495 data through successful trajectories, and (iv) adapting the framework to video creation and editing,
 496 where multi-tool orchestration across temporal dimensions presents additional challenges.